

SermVer

Arturo Silvelo

Try New Roads

SermVer

¿Por qué versionar?

Versionar un proyecto sirve para comunicar de forma clara y automática la naturaleza de los cambios.

Las versiones permiten identificar de forma inequívoca estados concretos del código (releases), facilitar la resolución de conflictos entre librerías y automatizar pipelines (builds, tests, publicación).

Ventajas de usar versionado semántico

- **Trazabilidad:** cada release queda marcado en el historial con un punto inmutable que se puede recuperar o desplegar.
- **Comunicación clara:** SemVer transmite expectativas sobre compatibilidad y esfuerzo de migración.
- **Gestión de dependencias:** facilita que consumidores y gestores de paquetes resuelvan versiones compatibles.

Versionado SemVer (Semantic Versioning)

- Formato: MAJOR.MINOR.PATCH
 - MAJOR: cambios incompatibles
 - MINOR: nuevas funcionalidades compatibles hacia atrás
 - PATCH: correcciones de bugs compatibles
- Ejemplo: 2.5.1 → mayor:2, menor:5, parche:1
- Convención recomendada: usa tags con prefijo **v** (ej. **v1.2.3**) para claridad

```
SemVer: <major>.<minor>.<patch>  
Ejemplos: v0.1.0, v1.0.0, v2.3.4
```

Otros Versionados

- Calendar Versioning (CalVer):
 - 2025.11, 2025.11.07
- Date-based:
 - 20251107
- Snapshot:
 - 1.2.0-SNAPSHOT
- Git-SHA:
 - 1.2.0+g7a3b2c

Versionado Git

SemVer define la semántica de la versión (MAJOR.MINOR.PATCH) y Git usa tags como marcadores inmutables en el historial para identificar y recuperar esas versiones concretas.

Tags en Git

- Crear tag anotado (recomendada):

```
git tag -a v1.2.3 -m "Release v1.2.3: descripción breve"
```

- Subir tag al remoto:

```
git push origin v1.2.3
```

- Listar tags:

```
git tag --list
```

- Obtener última etiqueta:

```
git describe --tags --abbrev=0
```


Flujo habitual para crear un tag manual (release)

1. Probar la aplicación

- Ejecutar test suite y checks (lint, build, pruebas de integración rápidas).

2. Actualizar versión en el proyecto (bump)

- JavaScript: `npm version patch|minor|major` (actualiza package.json + crea tag opcional).
- Maven: `mvn versions:set -DnewVersion=1.2.3` (actualiza pom.xml).
- Python: editar `pyproject.toml` / `setup.cfg` o usar herramientas como `bump2version`.

3. Actualizar CHANGELOG / notas de release

- Añadir resumen de cambios y breaking changes si procede.

4. Commit de los cambios de versión y changelog

```
git add <files>  
git commit -m "chore(release): bump version to v1.2.3"
```

5. Crear tag anotado que marque la release

```
git tag -a v1.2.3 -m "Release v1.2.3: resumen breve"
```

6. Push a remoto (branch + tag)

```
git push origin main  
git push origin v1.2.3  
# o enviar todos los tags:  
git push origin --tags
```

Flujo automático de releases con GitHub Actions

1. Trigger: push de tag `v*.*.*`.
2. Job CI: checkout → instalar deps → tests → build (wheel + sdist).
3. Job Release (needs: CI): crear Release en GitHub → subir artefactos (`dist/*`)
→ publicar en PyPI si está configurado el secret.

```
# Workflow: crear release automático al empujar tags SemVer (vX.Y.Z)
name: Release on tag

on:
  push:
    tags:
      - 'v*.*.*'

permissions:
  contents: write      # necesario para crear releases y subir assets
  packages: write      # necesario para publicar en registries (PyPI/GitHub Packages)

jobs:
  build-and-test:
    name: Build & Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install build deps
        run: |
          python -m pip install --upgrade pip
          pip install build pytest

      - name: Run tests
        run: |
          pytest -q

      - name: Build distributions
        run: |
          pip install build
          python -m build --sdist --wheel --outdir dist

      - name: Upload dist as job artifact
        uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/*
```

```
create-release:
  name: Create GitHub Release & Publish
  runs-on: ubuntu-latest
  needs: build-and-test
  steps:
    - name: Checkout (required for actions that read repo)
      uses: actions/checkout@v4

    - name: Download dist artifact
      uses: actions/download-artifact@v4
      with:
        name: dist
        path: dist

    - name: Create GitHub Release
      uses: actions/create-release@v1
      with:
        tag_name: ${github.ref_name}
        release_name: Release ${github.ref_name}
        body: |
          Releases notes for ${github.ref_name}:
          - Generated automatically by CI.
        draft: false
        prerelease: false
      env:
        GITHUB_TOKEN: ${secrets.GITHUB_TOKEN}

    - name: Upload release assets
      uses: actions/upload-release-asset@v1
      with:
        upload_url: ${steps.create-release.outputs.upload_url} || github.event.release.upload_url
        asset_path: dist
        asset_name: ${github.ref_name}-dist.tar.gz
        asset_content_type: application/gzip
      env:
        GITHUB_TOKEN: ${secrets.GITHUB_TOKEN}

    - name: Publish to PyPI (optional)
      if: ${secrets.PYPI_API_TOKEN != ''}
      uses: pypa/gh-action-pypi-publish@v1.5.0
      with:
        user: __token__
        password: ${secrets.PYPI_API_TOKEN}
        packages_dir: dist
```

Herramientas

Para cubrir el ciclo completo —versionado, creación de tags, generación de changelog y publicación de releases— existen herramientas que lo automatizan y estandarizan. Estas soluciones suelen:

- Actualizar la versión (bump) según las reglas (p. ej. Conventional Commits).
- Crear tags anotados que marcan el release.
- Generar changelog automáticamente a partir de los commits/PRs.
- Publicar la release y opcionalmente artefactos en registries (PyPI, npm, GitHub Packages).

Un paso clave es normalizar los mensajes de commit (commitlint / commitizen) para que la herramienta pueda calcular el siguiente versionado automáticamente.

Herramientas

- **semantic-release**: Gestión de versiones y publicación de paquetes totalmente automatizada.
- **Commitizen**: Herramienta potente para la gestión de releases que ayuda a los equipos a mantener mensajes de commit consistentes y significativos, a la vez que automatiza la gestión de versiones.

```
# NODE semantic-release
name: Release
on:
  push:
    branches:
      - main

permissions:
  contents: read

jobs:
  release:
    name: Release
    runs-on: ubuntu-latest
    permissions:
      contents: write
      issues: write
      pull-requests: write
      id-token: write
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version-file: ".nvmrc"

      - name: Install dependencies
        run: npm clean-install

      - name: Release
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        run: npm run release
```

```
# Python Commitizen
name: Release

on:
  push:
    branches:
      - main

jobs:
  bump-version:
    runs-on: ubuntu-latest
    name: "Bump version and create changelog with commitizen"
    steps:
      - name: Check out
        uses: actions/checkout@v5
        with:
          fetch-depth: 0
          token: "${ secrets.GITHUB_TOKEN }"
      - name: Create bump and changelog
        uses: commitizen-tools/commitizen-action@master
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          changelog_increment_filename: body.md
      - name: Release
        uses: softprops/action-gh-release@v1
        with:
          body_path: "body.md"
          tag_name: ${ env.REVISION }
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```


Monorepositorios

Un monorepo agrupa múltiples paquetes/proyectos relacionados en un único repositorio.

Ventajas

- Visibilidad: todos los proyectos en un solo sitio.
- Refactors atómicos: cambiar API y consumidores en un mismo commit.
- Reutilización: compartir utilidades y código fácilmente.
- Simplifica la coordinación entre equipos pequeños/medianos.

Inconvenientes

- Escalado: builds y CI pueden volverse costosos sin optimizaciones.
- Complejidad de tooling: hace falta orquestación (workspaces, caches, pipelines).
- Control de permisos y tamaño del repo pueden ser retos en organizaciones grandes.

Estrategias de versionado y releases

- Versionado fijo (single version): toda la repo comparte la misma versión (simple, coherente).
- Versionado independiente: cada paquete tiene su propia versión (flexible, más complejo).